

Speaker notes — 7.5 minutes

Graph-Based vs Transition-Based Dependency Parsing for Uzbek

A Cross-Architecture Study under Low-Resource Conditions

International Conference on Educational Data Mining

EDM 2026

Sanatbek Matlatipov, Maftuna Ismoilova, Mersaid Aripov

National University of Uzbekistan, Tashkent

`s.matlatipov@nuu.uz`

How to use these notes. Each numbered section is one slide. The bracketed times [**start** → **end**] are cumulative running time against a strict 7.5-minute budget. Bold phrases are the words to land on; [**pause**] is a deliberate beat. The prose is a spoken script — deliver it conversationally, not read verbatim.

Discipline for a short slot. Average ~30 s per slide and keep momentum. Do *not* read tables cell-by-cell — point to the one figure that carries the argument. Protect time on **Slide 7 (subword fusion)** and **Slide 10 (the 16.70 LAS gap)**; everything else can move briskly.

The single claim to leave behind: *for a morphologically rich, genuinely low-resource language, the choice of parsing architecture matters more than tagging accuracy — and cross-treebank data is the most powerful lever we have.* **Numbers to land: LAS 63.81 vs 47.11, a 16.70-point gap; and cross-treebank +9.62 LAS for the graph-based parser.**

Slide 1 — Title

[0:00 → 0:20 min]

Good morning, and thank you to the EDM organisers and to all of you for being here.

My name is **Sanatbek Matlatipov**, from the National University of Uzbekistan, presenting joint work with Maftuna Ismoilova and Mersaid Aripov. Our question is deceptively simple, and for Uzbek it has never been answered: **when you have almost no training data, which family of neural dependency parser should you actually reach for — graph-based or transition-based?** And, just as importantly, **why**.

Slide 2 — Why is parsing Uzbek hard?

[0:20 → 1:05 min]

Three properties make this hard. First, Uzbek is **agglutinative**: a single word stacks an ordered chain of suffixes encoding case, tense, person, even evidentiality. Second, it has **flexible subject–object–verb** word order with frequent center-embedding, plus null subjects and no articles — so surface cues English parsers rely on simply are not there. Third, and decisively, it is **low-resource**: the entire Universal Dependencies data for Uzbek is just **1,184 sentences**.

[pause]

The box on the right is the intuition that drives the whole talk. Take *bolalarning* — “of the children.” It decomposes into the stem *bola*, the plural *-lar*, and the genitive *-ning*. That final suffix is not decoration — it *is* the syntactic relation. So in Uzbek, **if you lose the last suffix, you lose the parse**. Hold onto that; it returns on Slide 7.

Slide 3 — Contributions

[1:05 → 1:35 min]

We make three contributions. First, the **first head-to-head comparison** of a graph-based parser, Stanza, against a transition-based parser, spaCy, for Uzbek — and critically, **both wired to the same TahrirchiBERT encoder**, so the architecture is the *only* variable. Second, we show that cross-treebank augmentation helps the two architectures **very differently** — a finding, not a footnote. Third, we give **per-relation and**

per-feature error analyses that explain *where* each architecture wins. Data, code, and checkpoints are all public.

Slide 4 — Two UD treebanks for Uzbek

[1:35 → 2:05 min]

We work with two treebanks. **UzUDT** — 684 sentences of fiction and academic prose, hand-annotated by six people on INCEpTION with inter-annotator agreement above 95%. And **Uzbek-UT** — 500 sentences of news and fiction, semi-automatic with full manual correction. Neither shipped with a standard split, so we re-split both consistently. Merged, that is **781 training sentences** — which roughly **doubles** the data over UzUDT alone. Keep that doubling in mind for the augmentation result later.

Slide 5 — Why the treebanks are complementary

[2:05 → 2:35 min]

But this is not just *more* data — it is **broader** data, and that distinction is the heart of the augmentation story. The two treebanks cover genuinely different syntax. UT is news-heavy: far more proper nouns and **ten times** the light-verb compounds. UzUDT is literary: **four times** the adverbial clauses, plus possessor agreement and evidentiality that UT simply does not annotate. So they fill each other's blind spots — the union covers more of the grammar than either alone.

Slide 6 — Two neural parsing architectures

[2:35 → 3:15 min]

Same encoder, two fundamentally different decoders. **Stanza** is graph-based: a two-stage pipeline ending in a Dozat–Manning biaffine parser that scores *every* possible head-dependent pair and then decodes the best tree globally. **spaCy** is transition-based: a single jointly-trained model that builds the tree incrementally with arc-eager shift–reduce moves. They even differ in how they collapse BERT sub-words into tokens — Stanza takes the last sub-word, spaCy averages. That sub-word choice turns out to matter, so let me take it next.

Slide 7 — Subword-to-token fusion

[3:15 → 4:05 min]

This is the slide to slow down on. BERT emits **WordPieces**, but Universal Dependencies annotates **whole tokens**, and the two do not align. So we must pool several sub-word vectors into one per token — and *how* we pool is a real modelling decision.

Look at the table. In every example the *last* piece — *##ning*, *##ni*, *##ib* — carries the grammatically decisive morpheme: genitive, accusative, converb. This is not a coincidence; it is a typological fact about a suffixing language.

[pause]

So **mean pooling** — spaCy’s default — averages every piece and *dilutes* exactly that suffix signal. **Last-sub-word** selection keeps it intact. The payoff is clean: in a controlled ablation, last-sub-word buys **+2.64 LAS** with **zero** extra parameters — and the gain shows up in *parsing*, not tagging, precisely because attachment in Uzbek is case-driven.

Slide 8 — Experimental setup

[4:05 → 4:30 min]

Briefly, the protocol. Seven models: five Stanza configurations, two spaCy. The suffix **point-one** means UzUDT only, **point-two** means merged. We deliberately use each framework’s **documented defaults** — this is a fair out-of-the-box comparison, not a tuning contest, which is how a practitioner actually meets these tools. One GPU, standard UD metrics: UPOS, XPOS, morphological features, and unlabeled and labeled attachment scores.

Slide 9 — Results within each framework

[4:30 → 5:05 min]

Two quick reads before the comparison. **Within Stanza**, on the left: contextual BERT beats static FastText on tagging, and the highlighted row — **E2.2**, BERT with last-sub-word on merged data — is our best configuration at **LAS 63.81**. **Within spaCy**, on the right: again, merged data wins. And notice the pattern that holds in *both* frameworks — the single biggest jump comes from merging, up to eleven LAS

points. That sets up the two questions the next slides answer.

Slide 10 — The central comparison

[5:05 → 5:55 min]

This is the core result of the paper. Same encoder, same merged data — **only the architecture differs**.

On tagging, spaCy is *better*: UPOS of 89 against 85. But look at the last row. On **labeled parsing**, Stanza scores **63.81** against spaCy's **47.11** — a **16.70-point LAS gap**. That is among the largest architecture gaps reported in the recent parsing literature.

[pause]

And here is the punchline I want you to take away: Stanza wins parsing *despite losing* on tagging. So for Uzbek, raw POS accuracy is *not* what drives parse quality — the **exhaustive biaffine search** is, because a rich case system and long-distance dependencies reward scoring all head-dependent pairs globally rather than committing greedily, left to right.

Slide 11 — Data augmentation is architecture-dependent

[5:55 → 6:25 min]

Now the second finding: merging helps both architectures, but **not in the same place**. For tagging the gains are nearly identical — about +2.6 UPOS either way. For *parsing*, they diverge sharply: Stanza gains **+9.62 LAS**, spaCy only +1.76 — the graph-based parser is the one that actually exploits the structural diversity we added. Symmetrically, spaCy's *morphology* leaps **+14.93**, because its joint objective feeds parser gradients back into the morphologiser. Architecture decides *what* extra data buys you.

Slide 12 — Per-relation error analysis

[6:25 → 6:55 min]

Where do the errors concentrate? The frequency plot on the left shows a heavily skewed relation distribution. The per-relation scores on the right tell the story: the transition-based parser is **strong on local, short relations** — *nummod*, *root* — but **weak on**

the long-distance ones: coordination, *conj*, drops to 18.9, and noun modification, *nmod*, suffers from the genitive *-ning* ambiguity we saw on Slide 2. These are exactly the structurally hard cases that exhaustive graph scoring handles better.

Slide 13 — Qualitative parses

[6:55 → 7:15 min]

Two real outputs from our best system to make this concrete. On the left, a **perfect parse** — 100% — with correct genitive–possessive chaining and a converb *xcomp*: the diagnostic Uzbek constructions, all right. On the right, a **systematic failure** — the light-verb construction *davom etmoq*, “to continue.” Gold *compound:lvc* is in blue; the model’s wrong arcs are dashed in red. Rare multi-word predicates like this are precisely what a tiny treebank under-covers — and precisely where more, broader data helps.

Slide 14 — Conclusion

[7:15 → 7:30 min]

To conclude. We find a clear **architecture-dependent trade-off**: transition-based parsing tags better, but graph-based parsing parses far better — **+16.70 LAS**. Cross-treebank augmentation is the **single most impactful lever**, and graph-based parsers exploit it best. So our practical recommendation for Uzbek and languages like it is unambiguous: **a graph-based parser, a monolingual BERT, and last-sub-word fusion**, whenever parse quality is the goal. Next steps are multi-seed significance testing, broader frameworks and encoders, and LLM-based data augmentation.

Slide 15 — Thank you

[7:30 → – min]

Thank you. The treebanks, the code, and all trained checkpoints are released — on Hugging Face and GitHub — and I would be glad to take your questions. (*Contact: s.matlatipov@nuu.uz.*)

Likely Q&A — quick cues

Q. Is the comparison fair if the hyper-parameters differ?

Yes, by design. We use each framework’s documented defaults to reflect the genuine out-of-the-box practitioner experience. On a corpus this small, tuning rarely flips an architectural ranking, and a 16.70-point LAS gap dwarfs any plausible tuning effect.

Q. Why does Stanza freeze BERT while spaCy fine-tunes it?

That reflects each framework’s **native training paradigm**, not a choice we imposed: Stanza trains a parser head over frozen features; spaCy fine-tunes the transformer jointly. We report both as practitioners would actually run them.

Q. These are single runs — is the 16.70 LAS gap real?

Yes. A paired-bootstrap on the 325-sentence test set gives a 95% confidence interval of about ± 2.4 LAS per model — far below the gap — and the ranking is monotone across all five Stanza runs. Multi-seed testing is our immediate next step.

Q. Why last-sub-word rather than mean or attention pooling?

It is **motivated by Uzbek morphology**: the rightmost sub-word carries the outermost, syntactically decisive suffix. Empirically it is +2.64 LAS over mean pooling with no extra parameters, and it should transfer to other suffixing Turkic languages.

Q. Why not UDPipe 2.0, Trankit, or XLM-R / UzbBERT?

This paper is a focused **graph-vs-transition** study, not a multi-framework benchmark. Broader frameworks and alternative encoders are explicit future work.

Q. Does merging help because of size or because of genre?

Both. It nearly doubles the data *and* the treebanks are genre-complementary — UT contributes proper nouns and light-verb compounds, UzUDT contributes adverbial clauses and possessor/evidential features.

Q. How does this connect to educational data mining?

Robust Uzbek parsing is foundational infrastructure for downstream educational NLP — automated writing feedback, readability assessment, and content analysis — for a language with 37 million speakers and very little tooling.